

SISTEMAS OPERACIONAIS · TURMA 02 · UFS

Processos, Threads e Inferência Local com IA Generativa

Ollama + Qwen3-8B rodando localmente: o que acontece por baixo do capô

Anderson Santana · Antônio Bispo · Antônio Mascarenhas · Davi Machado · João Felipe · José Gustavo · Kaique Rodrigues

IA generativa local não é só o modelo. Ela mobiliza processos, threads, memória, armazenamento, rede e mecanismos de proteção do SO.

Pergunta norteadora

Como a camada de aplicação, o modelo, a quantidade de parâmetros, a quantização e a configuração de execução afetam processos, threads, CPU, memória e a responsividade de um sistema local de IA generativa?

Trilha selecionada: A — Chat local

Open WebUI (container Docker) + Ollama (host, WSL2) + Qwen3-8B

O experimento em números

2

máquinas testadas

3

configurações comparadas

36

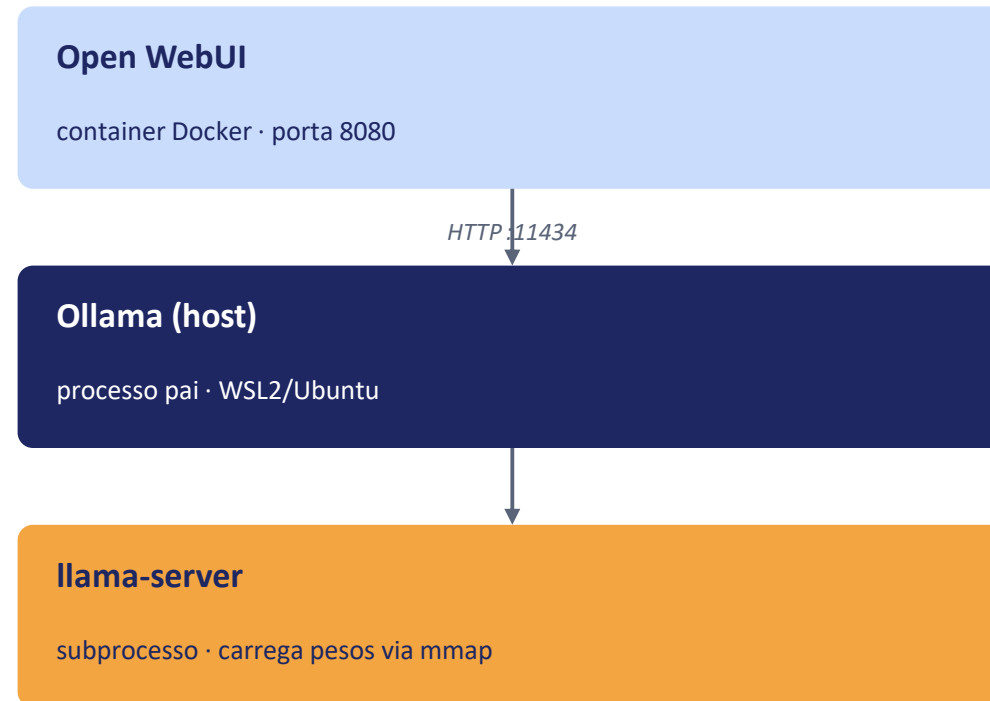
execuções mensuráveis

0%

taxa de erro

- Open WebUI em container Docker ↔ Ollama nativo no host Linux (WSL2/Ubuntu)
- Comunicação via API HTTP local: host.docker.internal:11434
- No Linux, esse endereço não resolve por padrão no Docker Engine (diferente do Docker Desktop)
- Docker Engine instalado direto no WSL/Ubuntu — essencial p/ o ps enxergar os processos do Open WebUI

Fix aplicado: `--add-host=host.docker.internal:host-gateway`



3

Inventário do Sistema

Antônio

Item	Máquina 1 (principal)	Máquina 2 (comparação)
Hostname	DESKTOP-B3LNN05	DESKTOP-1F600RV
SO	Ubuntu 26.04.1 LTS (WSL2)	Ubuntu 24.04.5 LTS (WSL2)
CPU	Ryzen 7 5700X (8 núcleos/16 th)	Ryzen 5 5600GT (6 núcleos/12 th)
RAM (WSL2)	11 GiB + 4 GiB swap	13 GiB + 4 GiB swap
Ollama	0.34.0	0.34.0

Modelo executado

Qwen3-8B · 8,19B parâmetros · GGUF Q4_K_M · ~4,9 GB em disco · contexto 32.768 tokens (até 131.072 com YaRN)

4 Processos e Threads

Kaique

ollama PID 1120 — processo pai

orquestrador / servidor HTTP :11434 · ~13 threads

cria (fork/exec)

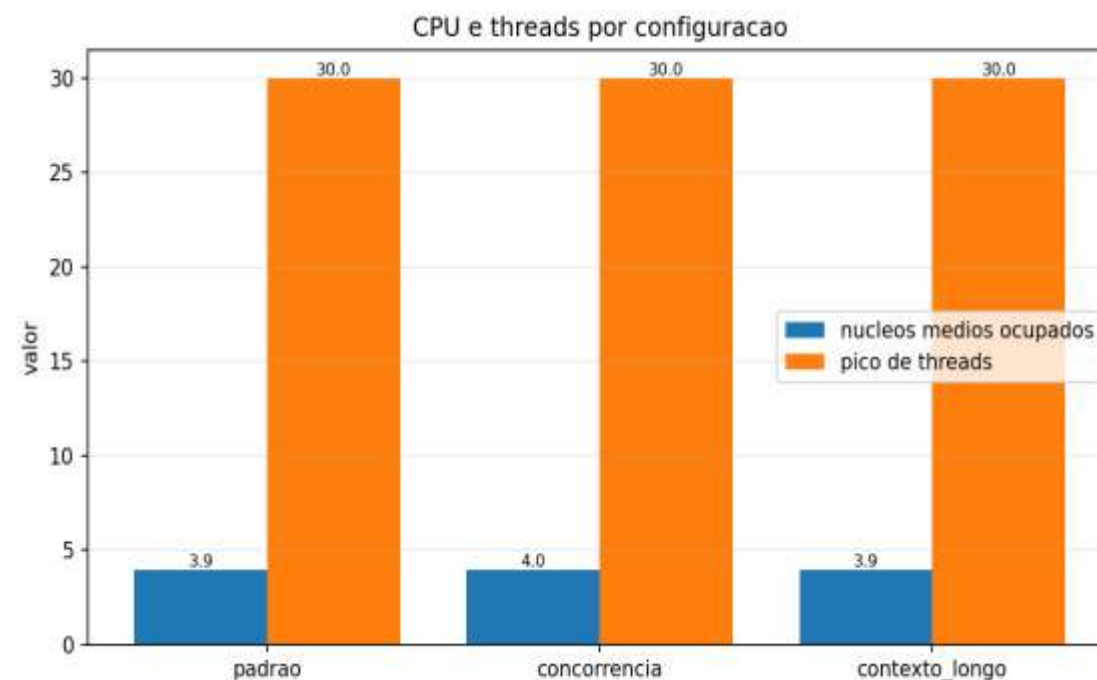
llama-server PID 1681 — filho de 1120

carrega pesos (mmap) + inferência · 10 threads de cálculo

open-webui (uvicorn) — container, porta 8080

processo isolado do host · 1 processo, workers=1

Threads = mesmo espaço de endereçamento do processo pai.



CPU e threads por configuração — núcleos ocupados praticamente saturados (~4) e pico de ~30 threads em todos os cenários.

Syscall	% do tempo (M1 / M2)	Interpretação
futex	85,4% / 90,0%	Sincronização entre threads de cálculo do llama-server (lock/wait/wake)
accept4	9,3% / 6,8%	Servidor HTTP do Ollama aceitando conexões do Open WebUI
read	4,5% / 2,6%	Leitura de dados de sockets (corpo das requisições HTTP) e arquivos
epoll_pwait / nanosleep	~0,5% / ~0,4%	Espera assíncrona por I/O e pausas curtas de threads worker

futex domina 85–90% do tempo total: as threads passam a maior parte do tempo bloqueadas aguardando trabalho ou a conclusão de etapas de cálculo — não é custo de processamento equivalente.

6

Experimentos e Automação

3 configurações comparativas

Padrão

num_ctx=4096 · 1 requisição por vez

Concorrência

num_ctx=4096 · 4 requisições simultâneas

Contexto longo

num_ctx=8192 · 1 requisição por vez

Cada config. × 2 tamanhos de prompt (curto ~51 tok / longo ~1.794 tok) × 3 repetições → 18 execuções/máquina

Automação com apoio de IA

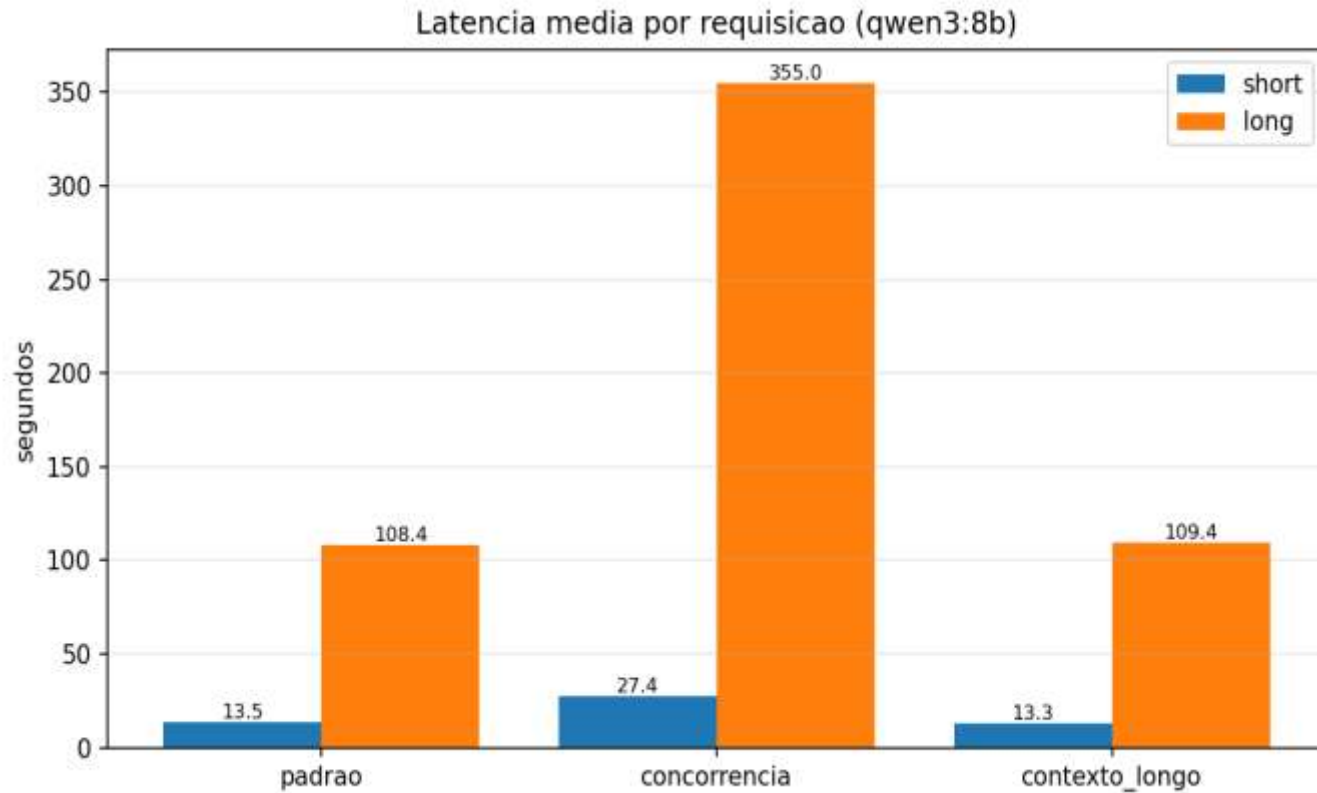
- Inventário do ambiente
- Setup (Ollama + Open WebUI)
- Monitoramento de processos
- Captura de syscalls (strace)
- Benchmark comparativo

5 scripts Bash, gerados e revisados com Claude Opus 5 — validados por execução real e conferência de logs/JSON antes de virar metodologia final.

0%

taxa de erro nas 36 requisições por máquina — todas HTTP 200, sem timeouts

7 Resultados — Latência e Vazão



Latência média por requisição (segundos) — Máquina 1

Curto: 12,65s → 25,20s

com 4 req. simultâneas (M2)

Longo: 94,16s → 300,65s

com 4 req. simultâneas (M2)

Vazão curto: +94%

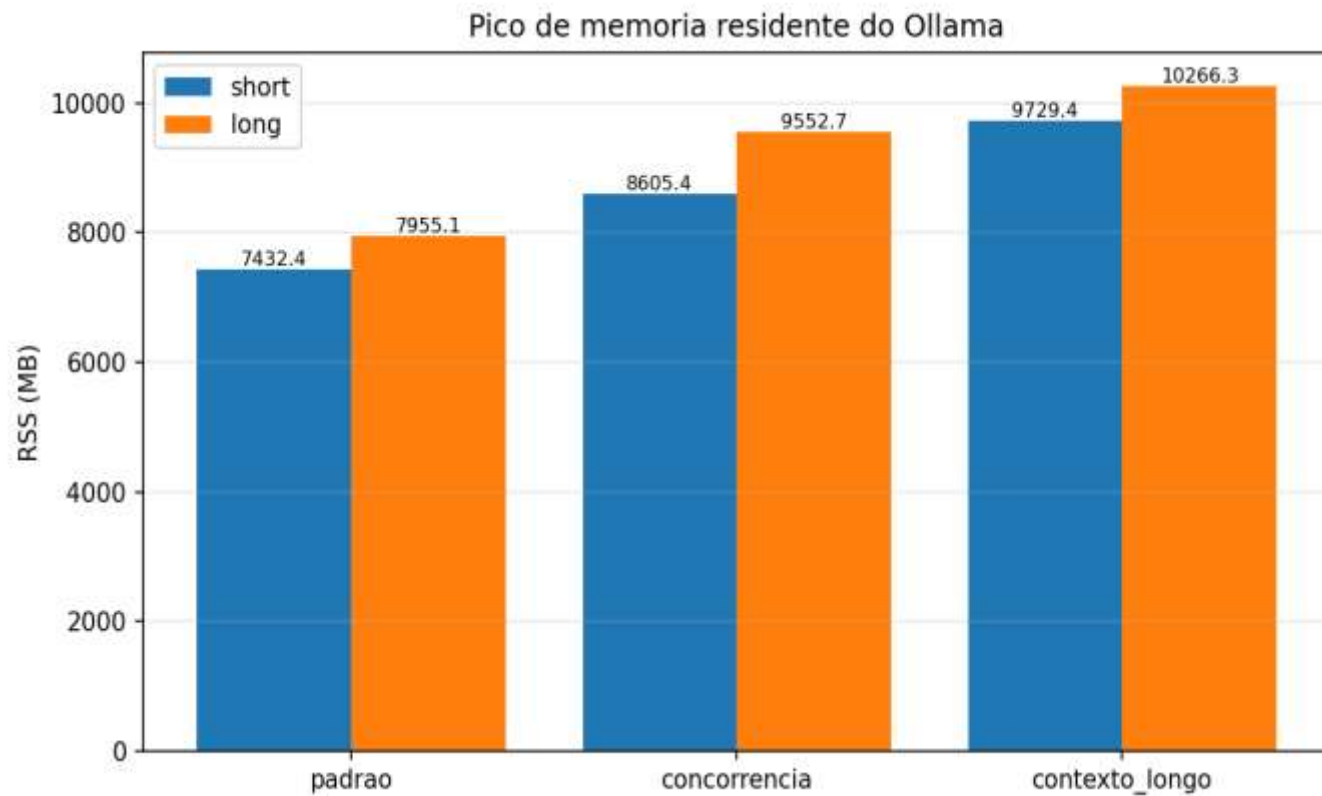
req/s sobe com concorrência

Vazão longo: +25%

ganho bem menor com prompt longo

8

Resultados — Memória e CPU



Pico de memória residente (RSS, MB) do Ollama por configuração

+2,3 GB de RSS

ao dobrar num_ctx (4096→8192), mesmo sem mudar o prompt — alocação antecipada

-45% CPU/req (curto)

e -18% (longo) sob concorrência — reaproveitamento de trabalho

CPU já saturada

~4 núcleos ocupados mesmo com 1 única requisição em andamento

Concorrência troca responsividade individual por capacidade agregada — não existe resposta única sobre se “compensa”.

Prompt longo domina o prefill

1,40s → 54,31s (39×) até o 1º token. É esse tempo que define a percepção de responsividade do usuário.

Contexto maior custa memória, não tempo

Dobrar num_ctx reserva cache de atenção antecipadamente — o custo é pago na carga do modelo, não sob demanda.

Execução local = privacidade e autonomia

Sem envio de dados a terceiros, mas com responsabilidade integral da equipe pela segurança da infraestrutura.

- A concorrência não melhora nem piora o desempenho de forma absoluta — ela troca latência individual por vazão agregada, e esse compromisso se deteriora com entradas longas
- O tamanho da entrada é o fator de maior impacto na experiência de uso (fator de 39× no tempo até a 1ª resposta)
- Os resultados evidenciam, na prática: escalonamento e concorrência, gerência de memória, comunicação entre processos e hierarquia pai-filho
- Limitações: truncamento em 256 tokens, virtualização WSL2 e 3 repetições por cenário

39×

no tempo até 1ª resposta (prompt longo)

+94%

de vazão com concorrência (prompt curto)

+2,3 GB

de RSS ao dobrar a janela de contexto

Obrigado!

Perguntas?